

RoboMemory: A Brain-inspired Multi-memory Agentic Framework for Interactive Environmental Learning in Physical Embodied Systems

Mingcong Lei^{1,3*}, Honghao Cai^{3*}, Yuyuan Yang^{3*}, Yimou Wu^{7*}, Jinke Ren^{1,2}
 Zezhou Cui^{1,3}, Liangchen Tan⁴, Junkun Hong¹, Gehan Hu^{1,3}, Shuangyu Zhu¹
 Shaohan Jiang^{1,3}, Ge Wang^{1,3}, Junyuan Tan¹, Zhenglin Wan⁵, Zheng Li⁶, Zhen Li^{1,2}
 Shuguang Cui^{1,2}, Yiming Zhao^{1,7}, Yatong Han^{1,7}

¹ FNii-Shenzhen ² SSE, CUHK-Shenzhen ³ The Chinese University of Hong Kong, Shenzhen

⁴ The University of Hong Kong ⁵ National University of Singapore

⁶ The Chinese University of Hong Kong ⁷ Ising AI

yiming_zhao@hrbeu.edu.cn hanyatong@cuhk.edu.cn

Project website: <https://sp4595.github.io/robomemory/>

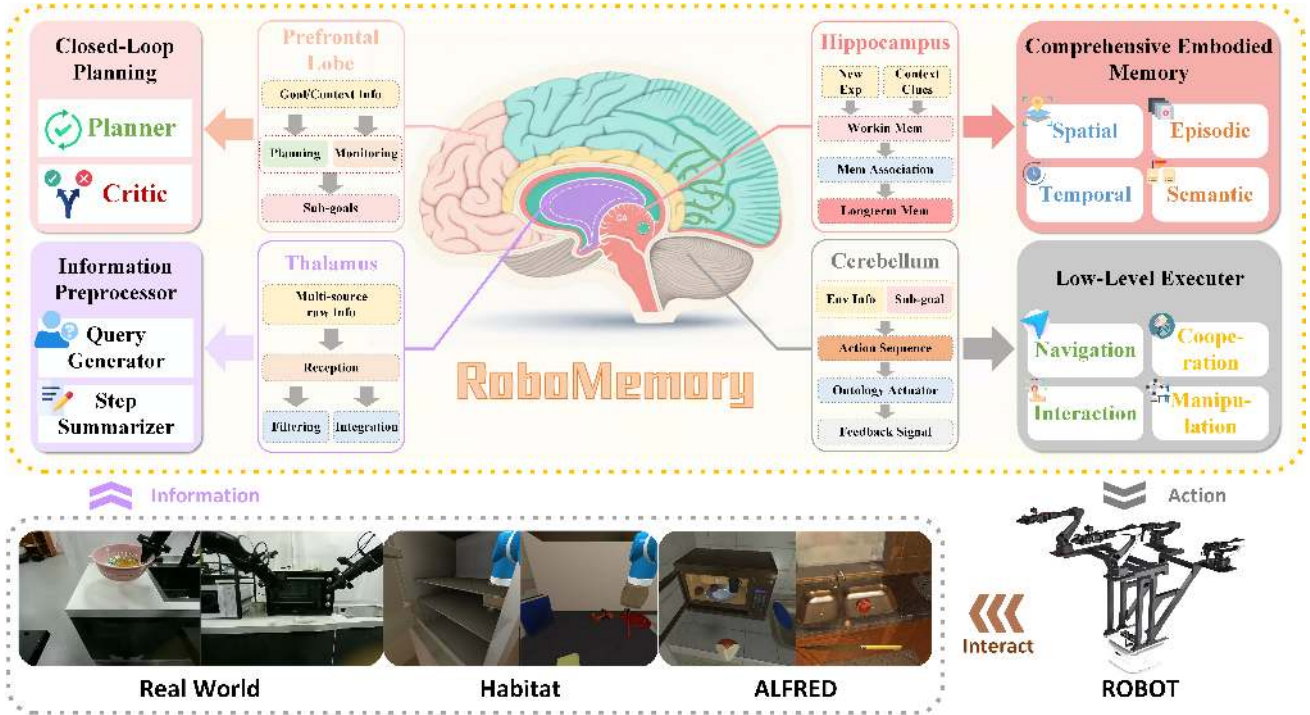


Fig. 1: RoboMemory adopts a brain-inspired architecture that maps neural components to agent modules, enabling long-term planning and interactive learning across diverse environments (real-world, Habitat, ALFRED) and robotic hardware.

Abstract—Embodied intelligence aims to enable robots to learn, reason, and generalize robustly across complex real-world environments. However, existing approaches often struggle with partial observability, fragmented spatial reasoning, and inefficient integration of heterogeneous memories, limiting their capacity for long-horizon adaptation. To address this, we introduce RoboMemory, a brain-inspired framework that unifies Spatial, Temporal, Episodic, and Semantic memory within a parallelized architecture for efficient long-horizon planning and

interactive learning. Its core innovations are a dynamic spatial knowledge graph for scalable, consistent memory updates and a closed-loop planner with a critic module for adaptive decision-making. Extensive experiments on EmbodiedBench show that RoboMemory, instantiated with Qwen2.5-VL-72B-Ins, improves the average success rate by 26.5% over its strong baseline and even surpasses the closed-source SOTA, Claude-3.5-Sonnet. Real-world trials further confirm its capability for cumulative learning, with performance consistently improving over repeated tasks. Our results position RoboMemory as a scalable foundation for memory-augmented embodied agents, bridging insights from

*Equal contribution

I. INTRODUCTION

Recent advances in Vision-Language Models (VLMs) [23, 6] have enabled their growing use in embodied tasks [33, 22]. VLM-based embodied agents can process multimodal inputs and generate high-level textual commands (e.g., “Pick up the cup”), which require translation via tool APIs to become executable robot actions. In contrast, Vision-Language-Action models (VLAs) [24, 8, 7, 13] produce low-level control signals directly but generally rely only on the latest observation. This limits their ability to perform long-horizon, multi-step tasks that require reasoning over task history. In summary, VLA models enable direct robot control but lack high-level planning capabilities, and VLM-based embodied agents support strategic planning but struggle with direct motor control. This highlights a key gap inherent in two distinct technical approaches to embodied intelligence.

To bridge this gap, recent work [50, 34, 38] proposes a “VLM planner + VLA executor” paradigm. Here, VLM-based embodied agents serve as high-level planners that decompose complex tasks (e.g., “make a coffee”) into executable sub-instructions (e.g., “grasp the cup”) that VLAs can complete. Although this paradigm improves performance on multi-step tasks, prior work suffers from two key limitations in real-world settings. First, real-world tasks (e.g., kitchen operations) require navigating across multiple locations to gather objects and tools, but the environment remains only partially observable at any time due to robots’ limited field of view and dynamic occlusions. This necessitates a planner with robust spatial awareness and long-term memory to maintain a consistent spatial awareness across viewpoints. However, most VLM-based agents rely on chat-style context windows (e.g., logging instruction – feedback pairs [49]), which lack mechanisms for maintaining an overview of the environment’s spatial layout. Consequently, agents cannot reliably track object locations or recognize previously visited states. Second, pretrained VLMs are rarely trained on embodied planning trajectories, especially long-horizon, spatially grounded ones. So VLM-based agents often struggle to generalize to real-world settings [47]. To overcome these challenges, VLM-based planners must support interactive environmental learning — the ability to acquire, integrate, and retrieve spatial, episodic, and semantic knowledge during task execution, thereby enabling adaptation through experience. In summary, the current VLM–VLA paradigm still lacks (1) a robust spatial and long-term memory mechanism to maintain a consistent view of the environment, and (2) an effective way to acquire and leverage embodied planning experience for generalization.

To address these challenges, we propose **RoboMemory**, a brain-inspired memory framework designed to bridge the gap between biological cognitive mechanisms and robotic embodied intelligence. Drawing on foundational cognitive theories, we recognize that human memory is inherently organized into a tiered architecture rather than a flat buffer: sensory memory captures transient perceptual inputs [37], short-term working

memory handles immediate information manipulation [4, 5], and long-term memory provides persistent storage for episodic events and semantic facts [3, 44]. Neuroscientific research further reveals that specialized brain regions underpin these functions: the thalamus integrates multimodal sensory inputs, the hippocampus consolidates spatial and episodic experiences into long-term storage [29], the prefrontal cortex regulates high-level planning based on retrieval, and the cerebellum coordinates low-level motor execution. Mimicking this biological hierarchy to ensure layered information processing, our recipe for this generalist memory system consists of four key stages (Figure 1): 1) **Information Preprocessing (Thalamus-inspired)**, which integrates multimodal sensory inputs for downstream processing; 2) **Comprehensive Embodied Memory (Hippocampus-inspired)**, which organizes experiential and spatial knowledge through a three-tier structure (sensory, short-term, and long-term). Crucially, we design four parallel-update modules (Spatial, Temporal, Episodic, and Semantic) to minimize latency while ensuring data consistency; 3) **Closed-Loop Planning (Prefrontal Cortex-inspired)**, where the planner utilizes retrieved memory to generate high-level action sequences; 4) **Low-level Execution (Cerebellum-inspired)**, which translates plans into precise robot actions using VLA models and SLAM-based navigation.

While recent frameworks [39, 18, 45, 1, 16, 54, 10] have integrated Retrieval-Augmented Generation (RAG) to enhance planning, they suffer from a critical limitation: most are tailored for simulated environments or rely on flat, text-based retrieval that fails to capture the spatial complexity of the physical world. These approaches often lack a hierarchical structure, leading to inefficiencies in long-term knowledge accumulation. Moreover, existing multi-module systems [38, 1] typically incur significant inference latency due to sequential memory updates. In contrast, our framework leverages a parallel-update paradigm and a hierarchical organization, enabling the agent to maintain a dynamic, spatially-aware worldview in real-time, which is essential for robust interaction in dynamic environments.

We evaluate RoboMemory on EmbodiedBench, a challenging long-horizon planning benchmark [47]. Using Qwen2.5-VL-72B as the base model, RoboMemory achieves state-of-the-art results, improving average success rates by **26.5%** over the baseline and outperforming the closed-source Claude3.5-Sonnet [2]. In real-world deployments, our system demonstrates the capacity for continuous improvement: by executing diverse tasks sequentially without memory resets, the agent effectively learns from environmental familiarization.

Our main contributions are three-fold:

- We propose **RoboMemory**, a brain-inspired, unified embodied memory system that integrates four parallel update modules into a single framework. It enables efficient, comprehensive memory operations and coherent knowledge integration, which are critical for interactive environmental learning in real-world embodied scenarios.
- We design a retrieval-based **incremental update algorithm** for the real-time evolution of Spatial Knowledge

Graphs (KGs). This algorithm bridges the gap between static knowledge representation and dynamic embodied interaction, establishing KGs as a practical and scalable modality for Spatial Memory. By retrieving relevant sub-graphs, detecting local inconsistencies, and merging new observations, our method ensures efficient maintenance and overcomes the scalability bottlenecks that previously hindered KG-based approaches.

- RoboMemory supports **interactive environmental learning in real-world environments**. It enables sequential diverse tasks without memory reset, with experience accumulation driving steady performance improvements, demonstrating practical long-term autonomous learning in physical scenarios.

II. RELATED WORK

A. VLM/LLM-based Agentic Frameworks in Embodied Tasks

The rapid advancement of VLMs/LLMs has led to diverse agent frameworks in embodied environments [49, 36, 26]. Embodied tasks involve partial observability and long-horizon planning, requiring memory systems to retain context. Some use time-ordered context buffers for short-term memory (due to VLMs/LLMs’ limited long-context processing) [49, 32]; others adopt experience buffers as long-term semantic memory [16, 35]. For long-duration tasks, skill libraries serve as procedural memory, with agents accumulating skills via interaction [45, 39]. However, in real-world settings, the low-level executor may fail to complete the task, making it challenging to construct a reusable, code-based skill library. So, explicit procedural memory still needs to be improved in real-world settings. Moreover, Recent efforts integrate diverse memories [51, 39, 1] but focus on virtual/GUI environments, leaving real-world multi-modal memory support for long-term planning under-explored.

B. Spatial Memory

Spatial information is critical for embodied agents. Prior work has explored three main representations, each with inherent limitations that motivate our high-level abstractions: **2D Semantic Maps** encode environments as pixel grids [51]. As the semantic maps are flattened, they discard 3D relationships (e.g., “above/beneath”) and precise geometry, limiting expressiveness for manipulation tasks requiring fine-grained spatial reasoning. **3D Point Clouds** preserve accurate 3D information [52]. However, a 3D point cloud only encodes low-level spatial information. We need to put in extra effort to help agents understand 3D Point Clouds. **Semantic Scene Graphs** [38, 28, 19, 48, 20] offering a *compact, high-level* representation directly amenable to planning. However, existing methods suffer from two key drawbacks: (1) Most previous methods adopt rigid tree-like structures, which are easy to update with new information, but cannot model non-hierarchical or cyclic spatial configurations. (2) Some previous methods try to use general graphs to store various spatial information. [20] However, their design often lacks efficient dynamic update mechanisms—adding or modifying relations

typically requires expensive full-graph recomputation, creating a scalability bottleneck for real-time embodied interaction.

For clarity, we summarize the differences among different memory systems in the previous work. The comparison is shown in Table III in the appendix. Additionally, we provide more ablation studies in Appendix A.

III. ROBOMEMORY

RoboMemory is a hierarchical embodied agent system that equips robots with three core memory capabilities: historical interaction logs, dynamically updated spatial layouts, and accumulated task knowledge. As illustrated in Figure 2, in each iteration, RoboMemory follows a process of “Perception – Memory – Retrieval – Planning – Execution” process, ensuring that the agent continuously calibrates its memory and behavior in dynamic environments.

First, the information preprocessor converts multimodal sensor inputs into a textual summary of the current scene, which serves as the primary input to the Comprehensive Embodied Memory. Next, the Comprehensive Embodied Memory updates its internal representations, including action histories, object locations, and experiential knowledge. After updating the information, the memory system retrieves contextually relevant entries to inform the Closed-Loop Planning Module. Then, leveraging this contextual memory, the Closed-Loop Planning Module generates high-level, text-based action instructions. Finally, these commands are dispatched to low-level executors, which directly control the robot and complete the instructions. The execution process is demonstrated in Algorithm 1 in the appendix.

A. Information Preprocessor

At each time step t , RoboMemory receives a visual observation $\mathcal{O}_t$: an RGB frame (in simulation) or a short video clip (on physical robots), representing the agent’s observations. Since raw visual data is unsuitable for direct use in memory construction and retrieval, RoboMemory first employs an information preprocessor to convert multimodal observations into textual representations, thereby providing a semantic interface for subsequent memory and planning modules. The information preprocessor executes two Vision-Language Models (VLMs) in parallel: (1) *Step summarizer* $\mathcal{S}$: It transforms $\mathcal{O}_t$ into a concise textual description s_t of the just-executed action. The string s_t is stored in the system’s working memory. (2) *Query generator* $\mathcal{Q}$: It derives a list of queries $q_t = [q_t^{(1)}, q_t^{(2)}, \dots, q_t^{(N)}]$ from the same observation $\mathcal{O}_t$. Each query $q_t^{(i)}$ is a natural language-based query. These queries are used to query information from the memory system that may be useful.

Together, $\mathcal{S}$ and $\mathcal{Q}$ provide a swift, text-based interface between raw sensory data and provide basic information in each iteration for RoboMemory’s Comprehensive Embodied Memory System.

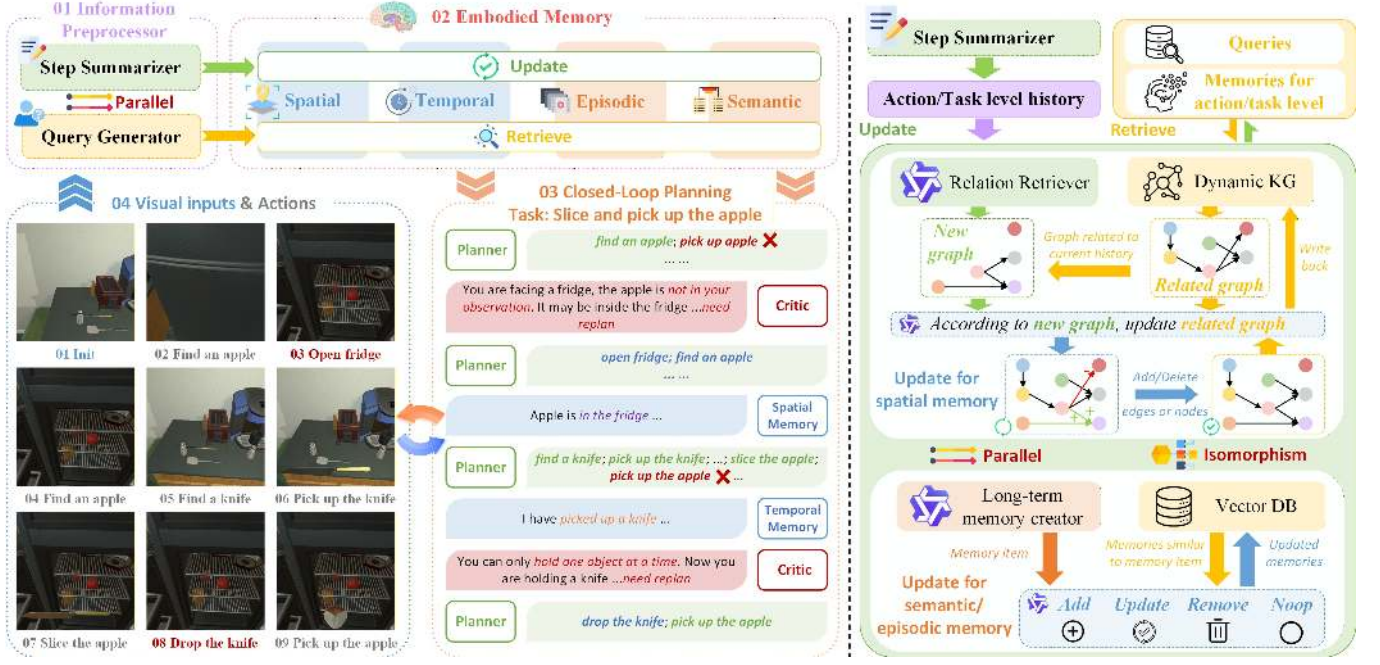


Fig. 2: (a) Left: The loop where the Planner, Critic, and Comprehensive Embodied Memory interact to adjust plans based on real-time visual inputs. Colored text denotes the execution status of actions (success/rejected/replanned). (b) Right: Spatial memory maintains a relevance/similarity-updated KG, and Semantic/Episodic memory manages a Vector DB with analogous logic. Besides, Temporal memory is implemented as a linear FIFO buffer that stores step-wise summaries generated by the Step Summarizer.

B. Comprehensive Embodied Memory

To address the long-term memory limitations in current embodied agent frameworks, we propose the Comprehensive Embodied Memory System. This system consists of multiple memory modules. We denote the memory system containing L distinct modules as $M_t = [M_t^{(1)}, M_t^{(2)}, \dots, M_t^{(L)}]$, where M_t represents the memory stored at step t , and $M_t^{(l)}$ denotes the l -th memory module. Generally, the update and retrieval process at iteration t of a general memory is shown below:

$$M_t = \mathcal{U}(M_{t-1}, s_t) \quad (1)$$

$$r_t = \mathcal{R}(M_t, q_t) \quad (2)$$

First, we update memory modules with memory update algorithm $\mathcal{U}$, where we update the previous memory M_{t-1} using the latest summarization s_t . Then, with updated M_t , we use memory retrieve algorithm $\mathcal{R}$ to retrieve the information that is useful for the planning module. In $\mathcal{R}$, we use queries q_t to query M_t , yielding retrieval results from each module: $r_t = [r_t^{(1)}, r_t^{(2)}, \dots, r_t^{(L)}]$. These results are then passed to the planning module, which helps it plan future movements. However, sequentially updating and retrieving from L modules would be a slow process. Therefore, we parallelize these steps across all modules, which significantly enhance the efficiency of the memory system.

The memory system consists of four distinct modules ($L = 4$): Temporal Memory, Spatial Memory, Semantic Memory, and Episodic Memory. For efficiency, all memory modules are updated and retrieved in parallel. Thus, even with multiple

modules, the system remains highly efficient. Functionally, inspired by cognitive psychology [27], our modules handle memory at different levels. In cognitive psychology, memory is divided into Sensory Memory, Short-term Memory, and Long-term Memory. Mirroring this hierarchy, our modules are organized as follows. First of all, the step summarizer $\mathcal{S}$ summarizes the agent's interactions with the environment at each iteration. It acts as Sensory Memory. Secondly, Temporal Memory and Spatial Memory function as short-term memory. These two memories will be updated at each iteration. They are designed to store the information of sensory memory in every iteration. For Temporal Memory, we record the agent's action history sequentially, while for Spatial Memory, we dynamically record the spatial relationships between different objects in the environment based on Sensory Memory. These memories can provide a relatively long and detailed history of the current task for the Closed-Loop Planner to make a future plan. Thirdly, Semantic Memory and Episodic Memory serve as Long-term Memory. They are updated only when meaningful information arises (e.g., after task completion). These memories store highly abstract knowledge, not limited to the current task, but synthesized from past experiences. This knowledge—factual, event-based, and experiential—improves the agent's future task performance. It is the source of RoboMemory's interactive learning capability. We now detail each module.

Temporal Memory. In the Temporal Memory, we record interactions between the robot and the environment (i.e., Sensory

Memory) of each iteration sequentially. This information can provide the embodied agent with simple awareness of “What I have done”. For such temporally sequential memory, a simple structure is sufficient: a sequential buffer with automatic summarization triggered when the record sequence reaches its capacity. In a specific design, temporal memory can store up to N interaction summaries, each generated by an information preprocessor. When the buffer is full, we compress the oldest N steps into a single summarized entry using a VLM, which is then reinserted at the front of the buffer, ensuring continuous context retention without unbounded growth. However, the information from previous memories will gradually be lost as we summarize it multiple times. For retrieval, we provide all existing memory in text to downstream modules.

Spatial Memory. The spatial memory is designed to dynamically record the high-level spatial relationships of different entities in the environment. However, current spatial memory approaches often rely on RGB-D cameras to reconstruct 3D point clouds [51, 9]. These representations are too detailed for high-level planning in embodied agents. For example, precise geometric relationships (e.g., exact distances between objects) are unnecessary.

To address these problems, we use a dynamic KG to store high-level spatial information: objects and positions in the environment become vertices of the KG, and spatial relations between objects or positions are encoded as edges. The KG focuses on high-level spatial relations (e.g., “cup on table”, “key left of drawer”). By these settings, spatial KG focuses on semantically meaningful, task-relevant relations. This spatial information enhances the agent’s spatial reasoning capability in dynamic environments.

However, as related work shows, most KG construction algorithms are designed for static long content. This does not meet the demand of using KG as spatial memory for the agent. The KG needs to update efficiently in response to new information. To address this issue, we introduce a retrieval-driven, incremental KG update algorithm that maintains a locally modifiable, globally consistent, and dynamically adaptive spatial memory. As illustrated in the right panel of Figure 2, the update process proceeds in four steps: (1) retrieves the most relevant sub-KG around new observations. (2) Injects new relations from the current observation by a VLM-based Relation Retriever. (3) Detects and resolves conflicts between newly extracted relations and existing ones (e.g., “cup on table” vs. “cup in drawer”) using a VLM-based resolver, which decides whether to add, delete, or modify edges. (4) Merges back and prunes isolated vertices. Moreover, our retrieval-based incremental update algorithm is accompanied by provable efficiency guarantees. For a KG with n vertices and maximum degree D , the number of vertices processed per update is bounded by $O(D^K)$, where K is the retrieval hop distance (see Appendix D for analysis). Further implementation details are provided in Appendix C1.

Semantic Memory. In cognitive psychology, semantic memory stores time-independent facts. These facts are stable, update slowly, and require long-term retention. In RoboMem-

ory, semantic memory records task-relevant experiences and environmental knowledge during execution. This information can help RoboMemory adapt to new environments or tasks. This information is highly abstract and does not need to be updated frequently. In RoboMemory, semantic memory updates when new information is encountered during execution, for example, after completing a subtask or encountering important information. To store and update memory efficiently, we design a memory management system based on a vector database (DB). In the vector DB, each experience/fact is described in natural language (denoted as a memory item). Each memory item is converted into a semantic vector for querying. For dynamic updates, we adapt a basic method from prior work [12]. However, we adopt it into the embodied environment. As shown in the bottom-right of Figure 2, the semantic memory update algorithm involves two VLM-based modules. Firstly, the Long-term Memory Creator generates new memory items based on short-term memory. We retrieve the top- S most similar existing memory items from existing memory via cosine similarity. A VLM-based updater then compares new and existing items to decide whether to: *add* the new item, *update* an existing item, *remove* an outdated item, or perform *Noop* (if redundant). Since updates only involve a maximum of S previous memory items and the update process is parallelized across all memory modules, this update method ensures that semantic memory remains efficient even as the database grows. For retrieval, we use the same process as a traditional vector DB. We use queries to extract top- N relevant information for downstream modules.

Episodic Memory. In cognitive psychology, Episodic Memory is another important part of long-term memory. It can store task-specific execution summaries (i.e., “autobiographical” records of past attempts). In RoboMemory, the Episodic Memory module is responsible for recording every interaction trajectory it has gone through, including the sequence of actions the robot did and the feedback from the environment. The trajectory information can help to improve the planning ability of RoboMemory. For example, if a trajectory for completing a similar task is stored in episodic memory, it can guide the agent in completing the current task. As the agent only needs to follow the successful trajectory in the memory, it can reduce hallucinations or errors in the VLM planner. Technically, Episodic Memory shares the same storage and VLM-driven vector DB update mechanism as Semantic Memory, ensuring consistent architectural design.

C. Closed-Loop Planning Module

The Closed-Loop Planning Module integrates information about the current task provided by the Spatial-Temporal Memory, Semantic and Episodic information recorded in long-term memory, and current observations to perform action planning. Each action is planned and passed on to the low-level executor for execution.

To enable closed-loop control in embodied environments, the Closed-Loop Planning Module adopts the Planner-Critic mechanism [25], which consists of the planner and the critic

module which is powered by same VLM model with different prompts. We denote the planner module as $\mathcal{P}$, while the critic module as $\mathcal{C}$. For each planning step, the planner generates a long-term plan consisting of multiple steps. However, due to the dynamics of embodied environments, the action sequence in the long-term plan may become outdated during the execution of the plan. Thus, before executing each step, we use the Critic model to evaluate whether the proposed action in this step remains appropriate under the latest environment. If not, the planner will re-plan based on the latest information. The demonstration of this process is shown in Figure 2.

However, our experiments reveal that the original Planner-Critic mechanism may suffer from infinite loops. In the original mechanism, the first step of the action sequence output by the Planner is evaluated by the Critic before execution, which can lead to an infinite loop: if the Critic always demands replanning, no action will ever be executed. To address this, we modify the Planner-Critic mechanism so that the first step is not evaluated by the Critic. This ensures that even if the Critic persistently demands replanning, the RoboMemory will still execute actions. The detailed algorithm is shown in Algorithm 1 in the appendix.

D. Low-Level Executor

The RoboMemory is a two-layer hierarchical agent framework that accomplishes longer-term tasks in the real world. The upper layer is responsible only for high-level planning, while the Low-level Executor carries out the actions planned by the upper layer in the real environment.

We employ a LoRA-finetuned VLA model, π_0 [21, 8], to generate manipulation actions, and a SLAM-based navigation model for locomotion. The low-level executor then translates high-level actions planned by RoboMemory into concrete arm and chassis movements in the real world.

IV. EXPERIMENTS

A. Benchmarks

To evaluate RoboMemory’s task planning ability, we select a subset of the EmbodiedBench EB-ALFRED and EB-Habitat benchmark [47]. We select the Base and Long subsets because they aim to test the agent’s planning ability. The Base and Long subsets of the two benchmarks comprise 200 tasks for complex embodied tasks. The EB-ALFRED and EB-Habitat benchmarks provide a visually grounded operational setting that closely mimics real-world conditions (see Appendix E for environment details), enabling direct comparison with established baselines. We set temperature = 0 to avoid randomness during the experiment, aligning with the EmbodiedBench setting. Each task is executed once. Moreover, we set up an environment to test the interactive environmental learning ability of RoboMemory in the real world.

B. Baselines and Metrics

To facilitate comparisons, we consider two types of baselines. First, we choose the advanced closed-source and open-source VLMs as a single agent. We compare their performance

with RoboMemory. For closed source VLMs, we choose GPT-4o and GPT-4o-mini [31, 23], Claude3.5-Sonnet and Claude-3.7-Sonnet [2], Gemini-1.5-Pro and Gemini-2.0-flash [41, 15]. For open source VLMs, we choose Llama-3.2-90B-Vision-Ins [30], InternVL-2.5-78B/28B [11], InternVL-3-72B [55], and Qwen2.5-VL-72B-Ins [6]. Secondly, we choose three agent frameworks: (1) Reflexion [35], which introduces a simple long-term memory and a self-reflection module. Reflexion uses the self-reflection module to summarize experiences as long-term memory, thereby enhancing the model’s capabilities. (2) Voyager [45], which utilizes a skill library as its procedural memory, is a widely used baseline for embodied agent planning. (3) Cradle [39], which proposes a general agent framework with episodic and procedural memory and gains good performances at various multi-model agent tasks. (4) RoboOS [38], which proposes an embodied agent framework that consists of a Hierarchy Scene-Graph based Spatial Memory.

In our experiments, each agent framework is tested using Qwen2.5-VL-72b-Ins [42] with temperature set as 0. For the RoboOS framework, we test it on RoboBrain2-32B [40], where the RoboBrain2-32B model is designed for the RoboOS framework.

The Qwen2.5-VL-72b-Ins represents a high-performing open-source alternative. Notably, the Qwen2.5-VL-72b-Ins demonstrates performance comparable to advanced closed-source VLMs in several benchmark tasks [46]. We use the Qwen3-Embedding model [53] to create embedding vectors for RAGs in RoboMemory. For the Low-level Executor, since EB-ALFRED provides high-level action APIs, we use the Low-level executor provided by EmbodiedBench instead of the VLA-based method.

We define two evaluation metrics to assess the performance: (1) Success Rate (SR), which is the ratio of completed tasks to the total number of tasks in each difficulty level. This metric reflects the agent’s ability to complete tasks across randomly generated scenarios. (2) Goal Condition Success Rate (GC), which is the ratio of intermediate conditions achieved to the maximum possible score in each scenario. An GC of 100% indicates that the task is completed in the given scenario. These two metrics can be computed as:

$$SR = \mathbb{E}_{x \in \mathcal{X}} [\mathbb{1}_{SCN_x = GCN_x}] \quad (3)$$

$$GC = \mathbb{E}_{x \in \mathcal{X}} \left[\frac{SCN_x}{GCN_x} \right] \quad (4)$$

In above formulas, $\mathcal{X}$ denotes the test subset, and x represents a test task. The success condition number (SCN_x) refers to the number of conditions the agent has accomplished, while the global condition number (GCN_x) indicates the total number of conditions required for task completion. The task is considered successful if $SCN_x = GCN_x$.

C. Main Results

As shown in Table I, our model achieves significant improvements over both single VLM agents and Agent frameworks on the EB-ALFRED and EB-Habitat. Compared to the SOTA Single VLM-Agent model, Claude3.5-Sonnet,

TABLE I: Comparison of Success Rates (SR) and Goal Condition Success Rates (GC) across difficulty levels (Base/Long) on EB-ALFRED and EB-Habitat benchmarks. Values are reported in percentages (%).

Method	Type	EB-ALFRED						EB-Habitat			
		Average		Base		Long		Base		Long	
		SR	GC	SR	GC	SR	GC	SR	GC	SR	GC
Single VLM-Agents											
GPT-4o	Closed-source	67.0	74.9	64.0	74.0	54.0	62.5	86.0	90.7	64.0	72.2
GPT-4o-mini		30.5	40.9	34.0	47.8	0.0	17.0	74.0	77.5	14.0	21.3
Claude-3.7-Sonnet		68.5	-	68.0	-	70.0	-	90.0	-	46.0	-
Claude-3.5-Sonnet		69.5	71.8	72.0	72.0	52.0	54.5	96.0	97.5	58.0	63.3
Gemini-1.5-Pro		68.0	73.3	70.0	74.3	58.0	65.0	92.0	92.5	52.0	61.2
Gemini-2.0-flash		57.0	61.5	62.0	65.7	58.0	62.0	82.0	82.0	26.0	36.2
Llama-3.2-90B-Vision-Ins	Open-source	40.5	46.6	38.0	43.7	16.0	24.0	94.0	94.5	14.0	24.3
InternVL2.5-78B		47.0	52.9	38.0	42.3	42.0	49.0	80.0	82.0	28.0	38.2
InternVL2.5-38B		37.5	42.6	36.0	37.3	26.0	36.5	60.0	61.5	28.0	35.0
InternVL3-78B		49.5	-	38.0	-	36.0	-	84.0	-	40.0	-
Qwen2.5-VL-72B-Ins		44.0	-	50.0	-	34.0	-	74.0	-	18.0	-
VLM-Agent Frameworks											
Voyager (Qwen2.5-VL-72B-Ins)	Baselines	46.5	66.4	56.0	73.2	32.0	54.2	76.0	87.0	22.0	51.0
Reflexion (Qwen2.5-VL-72B-Ins)		38.3	51.1	48.0	54.0	10.0	33.0	80.0	84.2	15.0	33.0
Cradle (Qwen2.5-VL-72B-Ins)		44.5	57.0	54.0	67.9	32.0	41.0	62.0	67.0	30.0	52.1
RoboOS (Qwen2.5-VL-72B-Ins)		25.5	33.0	32.0	38.4	12.0	17.6	38.0	47.8	20.0	28.2
RoboOS (RoboBrain2-32B)		20.0	25.7	32.0	37.2	8.0	13.2	28.0	34.8	12.0	17.4
RoboMemory (Qwen2.5-VL-72B-Ins)	Ours	70.5	79.7	68.0	75.5	66.0	81.3	86.0	88.0	62.0	74.0

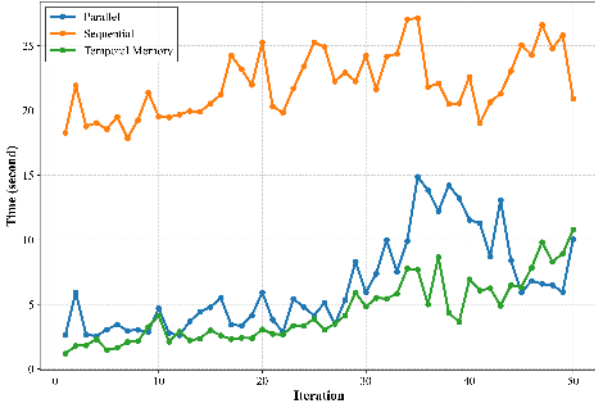


Fig. 3: Efficiency improvement of Comprehensive Embodied Memory System.

RoboMemory with Qwen2.5-VL-72B-Ins backbone improves the average SR by 1% and GC by 7.9%. This demonstrates RoboMemory’s superiority over single VLM-Agents, proving that an Agent framework with open-source models can outperform closed-source SOTA models. Furthermore, when tested against other VLM-Agent frameworks, RoboMemory also shows substantial gains. This is because, unlike other agent frameworks, RoboMemory’s brain-like memory system provides embodied models with more accurate and persistent contextual information. Additionally, the Planner-Critic mechanism provides a closed-loop planning ability, which helps the RoboMemory gain better performance in long-term tasks. Because the RoboMemory can detect and try to overcome possible failures. And it is more robust when encountering unexpected situations.

D. Efficiency Analysis

To evaluate the efficiency of the Comprehensive Embodied Memory module, we tested RoboMemory with executing 10 long-horizon tasks, each comprising approximately 50 steps. All testing is under the same hardware and software settings. We exclusively measured the wall-clock time consumed by memory update and retrieval operations. We analyzed the scaling behavior of memory update latency across three distinct configurations: (1) fully parallel update and retrieval across all memory modules; (2) sequential update of each memory module without parallelism; and (3) update of only the most fundamental memory component: the Temporal Memory. Results are presented in Figure 3. As shown, our parallel update strategy enables updating a multi-module memory system with latency comparable to that of updating a single base memory module. This demonstrates the critical efficiency gains afforded by parallelization across the memory architecture.

TABLE II: Ablation study of RoboMemory’s Success Rate (SR) on EB-ALFRED subset.

Method (EB-ALFRED)	Avg.	Base	Long
RoboMemory	67%	68%	66%
- w/o critic	55 %	60 %	50%
- w/o spatial memory	47 %	52 %	42 %
- w/o episodic memory	62%	68%	56%
- w/o semantic memory	58%	66%	50%
- w/o long-term memory	57%	66%	48%



Fig. 4: Visualization of the experimental environment.

E. Ablation Studies

We used the full Base and Long Subset from EB-ALFRED to validate RoboMemory’s effectiveness. We removed each component systematically and observed performance changes across task categories. We use the success rate as our metric. Results are shown in Table II.

Long-term Memory: Adding long-term memory significantly improves RoboMemory’s success rate. The experiment shows that it enables interactive environmental learning while attempting to complete tasks. Semantic memory learns the properties of low-level skills, such as in what circumstances an action may fail. The episodic memory records all task attempts (successful/failed), providing valuable experience at the task level and giving insight into how to complete a task successfully. This helps the RoboMemory predict the outcomes of actions and avoid ineffective attempts. This ability indicates that the RoboMemory has an interactive environmental learning capability. Additionally, we do ablations for semantic memory and episodic memory solely. The result shows that those memory modules significantly improve the capability of the performance of RoboMemory.

Spatial Memory: Spatial memory is crucial for embodied agents, especially given that current pretrained VLMs have limited spatial understanding ability. Our novel dynamic KG update algorithm enables KG-based spatial memory in dynamic environments. This spatial reasoning helps RoboMemory handle partially observable embodied settings.

Critic Module: Table II shows performance without the critic module (55% vs 67% with full system). This drop highlights how the critic’s closed-loop planning adapts to dynamic environments. It helps RoboMemory recover from failures faster and handle unexpected situations better.

F. Real-World Robot Deployment

To evaluate RoboMemory’s interactive environmental learning capability in the real world, we designed a kitchen environment inspired by EB-ALFRED and EB-Habitat. The scene contains 5 navigable points, 8 interactive objects, and

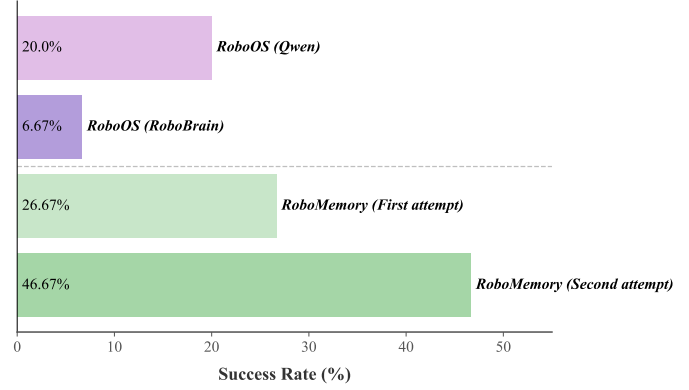


Fig. 5: Real-world experiment results. “Qwen” denotes Qwen2.5-VL-72B-Ins; “RoboBrain” denotes RoboBrain2.0-32B.

over 10 non-interactive (but potentially distracting) items. The environment is shown in Figure 4. In the real world, we use interactive environmental video recordings captured during action execution (rather than static snapshots taken after action completion) as RoboMemory’s input. This provides a more temporally coherent perception. We created three task categories (5 tasks each). Tasks are matched to EB-ALFRED’s Base subset (avg. oracle: 10–20 steps), though actual executions often exceed 20 steps due to search and error recovery. Due to search and error recovery, the robot often exceeds 20 steps per task. Additional hardware experiment details are in Appendix E.

To test the interactive environmental learning ability of RoboMemory, we ran each task twice without clearing long-term memory between attempts. Meanwhile, we compared RoboMemory against previous SOTA on real-world experiments RoboOS as baselines. The success rates for first and second attempts and different settings of RoboOS are shown in Figure 5.

The second attempt showed significantly higher success rates. This proves RoboMemory’s long-term memory effectively guides subsequent tasks in real embodied environments. Key observations include: (1) Closed-loop error recovery: RoboMemory retries failed actions when possible, even if the low-level executor (VLA model) fails. (2) Spatial reasoning: RoboMemory remembers object locations and spatial relationships using its memory. (3) Interactive environmental learning: RoboMemory analyzes failure causes reasonably. These analyses guide future decisions. Detailed examples demonstrating these capabilities and further discussions are provided in Appendix F.

Moreover, we observe a significant drop in task success rates when deploying the agent with the Low-level Executor in real-world environments. This performance degradation primarily stems from the executor’s inherent limitations: (1) The VLA model exhibits unreliable instruction-following capabilities, frequently failing during grasping actions or selecting incorrect objects; (2) Pre-trained VLM models demonstrate inadequate

video understanding capability. They are struggle to interpret dynamic visual information such as action failures or state changes. These limitations collectively contribute to the reduced performance compared to simulated environments.

V. CONCLUSION

This paper propose RoboMemory, a brain-inspired multi-memory framework, facilitating long-horizon planning and interactive environmental learning in real-world embodied systems by addressing key challenges such as memory latency, task correlation capture, and planning loops. Experiments on EmbodiedBench demonstrate that RoboMemory outperforms state-of-the-art closed-source VLMs and agent frameworks, with ablation studies confirming the critical roles of the Critic module and spatial/long-term memory. Real-world deployment further validates its interactive learning capability through improved success rates in repeated tasks. Despite limitations arising from reasoning errors and executor dependence, RoboMemory provides a foundation for generalizable, memory-augmented agents, with future work aimed at refining reasoning and enhancing execution robustness.

REFERENCES

- [1] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164*, 2024.
- [2] Anthropic. Claude 3.5 sonnet, 2024. URL <https://www.anthropic.com/news/claude-3-5-sonnet>.
- [3] Richard C Atkinson and Richard M Shiffrin. Human memory: A proposed system and its control processes. In *Psychology of learning and motivation*, volume 2, pages 89–195. Elsevier, 1968.
- [4] AD Baddeley and GJ Hitch. Working memory: Past, present... and future. *The cognitive neuroscience of working memory*, pages 1–20, 2007.
- [5] Alan Baddeley. Working memory. *Memory*, pages 71–111, 2020.
- [6] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [7] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [8] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [9] Matthew Chang, Theophile Gervet, Mukul Khanna, Sri-ram Yenamandra, Dhruv Shah, So Yeon Min, Kavitha Shah, Chris Paxton, Saurabh Gupta, Dhruv Batra, et al. Goat: Go to any thing. *arXiv preprint arXiv:2311.06430*, 2023.
- [10] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning. *Advances in Neural Information Processing Systems*, 37:589–631, 2024.
- [11] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024.
- [12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [13] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [14] Wonje Choi, Jinwoo Park, Sanghyun Ahn, Daehee Lee, and Honguk Woo. Nesyc: A neuro-symbolic continual learner for complex embodied tasks in open domains. *arXiv preprint arXiv:2503.00870*, 2025.
- [15] Google DeepMind. Introducing gemini 2.0: our new ai model for the agentic era, 2024. URL <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>.
- [16] Dayuan Fu, Biqing Qi, Yihuai Gao, Che Jiang, Guanting Dong, and Bowen Zhou. Msi-agent: Incorporating multi-scale insight into embodied agents for superior planning and decision-making. *arXiv preprint arXiv:2409.16686*, 2024.
- [17] Zipeng Fu, Tony Z Zhao, and Chelsea Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.
- [18] Marc Glocker, Peter Hönig, Matthias Hirschi, and Markus Vincze. Llm-empowered embodied agent for memory-augmented task planning in household robotics. *arXiv preprint arXiv:2504.21716*, 2025.
- [19] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5021–5028. IEEE, 2024.
- [20] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- [21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

- [22] Yafei Hu, Quanting Xie, Vidhi Jain, Jonathan Francis, Jay Patrikar, Nikhil Keetha, Seungchan Kim, Yaqi Xie, Tianyi Zhang, Hao-Shu Fang, et al. Toward general-purpose robots via foundation models: A survey and meta-analysis. *arXiv preprint arXiv:2312.08782*, 2023.
- [23] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [24] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [25] Mingcong Lei, Ge Wang, Yiming Zhao, Zhixin Mai, Qing Zhao, Yao Guo, Zhen Li, Shuguang Cui, Yatong Han, and Jinke Ren. Clea: Closed-loop embodied agent for enhancing task execution in dynamic environments. *arXiv preprint arXiv:2503.00729*, 2025.
- [26] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems*, 36, 2024.
- [27] Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, et al. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*, 2025.
- [28] Joel Loo, Zhanxin Wu, and David Hsu. Open scene graphs for open-world object-goal navigation. *The International Journal of Robotics Research*, page 02783649251369549, 2025.
- [29] James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419, 1995.
- [30] Meta. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models, 2024. URL <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- [31] OpenAI. Gpt-4o mini: advancing cost-efficient intelligence, 2024. URL <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- [32] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [33] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [34] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- [35] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [36] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3009, 2023.
- [37] George Sperling. The information available in brief visual presentations. *Psychological monographs: General and applied*, 74(11):1, 1960.
- [38] Huajie Tan, Xiaoshuai Hao, Cheng Chi, Minglan Lin, Yaoxu Lyu, Mingyu Cao, Dong Liang, Zhuo Chen, Mengsi Lyu, Cheng Peng, et al. Roboos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. *arXiv preprint arXiv:2505.03673*, 2025.
- [39] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.
- [40] BAAI RoboBrain Team, Mingyu Cao, Huajie Tan, Yuheng Ji, Xiansheng Chen, Minglan Lin, Zhiyu Li, Zhou Cao, Pengwei Wang, Enshen Zhou, et al. Robobrain 2.0 technical report. *arXiv preprint arXiv:2507.02029*, 2025.
- [41] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burrell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [42] Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- [43] Endel Tulving. *Elements of Episodic Memory*. Oxford University Press, Oxford, GB, 1983.
- [44] Endel Tulving et al. Episodic and semantic memory. *Organization of memory*, 1(381-403):1, 1972.
- [45] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [46] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddhartha Naidu, et al. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314*, 2024.
- [47] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, et al. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025.
- [48] Yuncong Yang, Han Yang, Jiachen Zhou, Peihao Chen, Hongxin Zhang, Yilun Du, and Chuang Gan. 3d-mem: 3d scene memory for embodied exploration and reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17294–17303, 2025.
- [49] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [50] Haoqi Yuan, Yu Bai, Yuhui Fu, Bohan Zhou, Yicheng Feng, Xinrun Xu, Yi Zhan, Börje F Karlsson, and Zongqing Lu. Being-0: A humanoid robotic agent with vision-language models and modular skills. *arXiv preprint arXiv:2503.12533*, 2025.
- [51] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinzhong Zhou, Yilun Du, Joshua B Tenenbaum, Tianmin Shu, and Chuang Gan. Building cooperative embodied agents modularly with large language models. *arXiv preprint arXiv:2307.02485*, 2023.
- [52] Lingfeng Zhang, Xiaoshuai Hao, Yingbo Tang, Haoxiang Fu, Xinyu Zheng, Pengwei Wang, Zhongyuan Wang, Wenbo Ding, and Shanghang Zhang. *nava*³: Understanding any instruction, navigating anywhere, finding anything. *arXiv preprint arXiv:2508.04598*, 2025.
- [53] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- [54] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- [55] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

A. Additional Related Work

1) *Memory Systems in Embodied Intelligence: From Cognitive Theories to Robotic Implementations.*: Embodied agents depend on memory to integrate experiences, resolve partial observability, and support long-horizon planning—capabilities rooted in cognitive psychology. This section links foundational memory theories to robotic implementations, highlighting gaps addressed by subsequent work.

Cognitive Psychology-Inspired Memory Frameworks. Cognitive psychology provides hierarchical memory models for agent design. The multi-store model [3] formalizes sensory, short-term, and long-term memory tiers. Tulving [44, 43] refined long-term memory into episodic (event-specific) and semantic (factual) subtypes. The working memory model [5] extends short-term memory with specialized subcomponents for active information manipulation. Most robotic frameworks only partially adopt these hierarchies. Early works [14, 54] lacked full sensory-short-long-term structures, leading to inefficient knowledge accumulation. Recent frameworks [1, 39] incorporated episodic and semantic memory but treated them as independent buffers, ignoring dynamic tier interactions [29] observed in human memory. This disconnect between cognitive theories and robotic implementations limits agents’ ability to adapt to real-world dynamic environments.

Multi-Modal Memory in Embodied Agents. Contemporary frameworks face real-world limitations. Voyager [45] used code-based skill libraries that fail to generalize to executor errors. Reflexion [35] lacked spatial memory and relied on sequential updates, incurring high latency. RoboOS [38] integrated scene-graph spatial memory but omitted episodic memory and suffered from rigid graph structures. Existing multi-module memory systems [16, 10] process updates sequentially, leading to inference delays incompatible with real-time interaction. This contrasts with human memory efficiency, where sensory inputs are parsed and stored in parallel across specialized neural pathways [37]. Additionally, most frameworks [39, 1] are limited to simulated environments, lacking support for real-world multi-modal memory integration.

B. Additional Experiments

1) *Error Analysis*: We summarize the common errors of RoboMemory in the previous experiments. We classify errors into three main types: planning errors, reasoning errors, and perception errors.

The planning errors occur when the planner fails to generate correct actions. The reasoning errors occur when the planner and critic cannot properly process input information (including current observations and memory), even when the input is correct. Perception errors occur when incorrect information is provided to the planner-critic module.

We analyze RoboMemory trajectories for failed tasks. We identify error types based on the above definitions. A single task may contain multiple errors. We calculate the occurrence probability of each error type to show RoboMemory’s strengths and weaknesses. The results are shown in Figure 6.

We can observe that among all error types, the planning errors are the most common. This means that even though the memory modules can provide comprehensive information about the RoboMemory agent’s previous experience and spatial and temporal memory for the current task, the planner module may still not provide good action plans. This may be due to the capability of the pretrained base model.

The most common perception error is the hallucination error. We can observe that although some hallucinations can be handled by the critic module or memory information, there are still some cases in which the planner ignores all insights from memory and critic and fails to complete the task.

The detailed examples and discussions are provided in Appendix F.



Fig. 6: The reason why RoboMemory failed to complete the task

2) *Additional Efficiency analysis for Dynamic Spatial KG update algorithm*: We analyze the evolution of the spatial KG during long trajectories in EB-ALFRED, focusing on the first 20 iterations (with 95% confidence intervals). As shown in Figure 7, the total number of spatial relationships in the KG (red line) increases gradually over iterations as RoboMemory is exploring the environment. In contrast, the number of relationships retrieved for update at each iteration (blue line) remains relatively stable, typically ranging around 10 edges per iteration. This stability is achieved because our method only updates a local subgraph relevant to the current observation.

We define the retrieval ratio as the proportion of relationships updated at each iteration relative to the total number of relationships in the KG. As shown in Figure 7, this ratio (illustrated by gray bars) decreases steadily from 76% initially to 28% at iteration 20. This trend indicates that, as the KG

TABLE III: Comparison of Memory-Related Methods in Embodied Agent

Method	Multimodal	Episodic	Semantic	Spatial	Temporal	Procedural	Memory Implementation	Real Robot
NeSyC [14]	✓		✓		✓		Symbolic logic rules	✓
Reflexion [35]			✓		✓		Buffer	
Voyager [45]						✓	RAG	
MSI-Agent [16]			✓		✓		Database, RAG	
CoELA [51]	✓	✓	✓			✓	Top-down semantic map	
Cradle [39]	✓	✓			✓	✓	RAG	
Agent-S [1]	✓	✓	✓		✓		RAG	
Expel [54]			✓		✓		Buffer	
AutoManual [10]		✓	✓			✓	Buffer	
HiRobot [34]	✓						/	✓
Being-0 [50]	✓	✓			✓		Buffer	✓
RoboOS [38]	✓			✓	✓		Scene graph, database	✓
RoboMemory (Ours)	✓	✓	✓	✓	✓		RAG,KG	✓

grows, each update affects a progressively smaller fraction of the entire graph. This demonstrates that our spatial KG update mechanism effectively localizes modifications, ensuring computational efficiency and mitigating interference through context-aware incremental updates.

C. Dynamic Spatial Memory Update Algorithm

1) *Detailed Algorithm of Spatial KG Update:* Spatial Memory is a dynamically updated KG-based module designed to overcome agents’ limitations in spatial reasoning. Specifically, our Spatial Memory is formulated as a directed KG $G = (V, E)$, where V denotes the set of all objects in the environment. Each object is a vertex in the KG. Each object’s name is encoded into a single semantic embedding vector via a pretrained embedding model [53]. The edge set E captures spatial relationships between objects, each represented as a triple (e.g., $[obj_1, \text{relationship}, obj_2]$). To update the Spatial KG, we need to continuously extract new relationships from current observations and update the old relationships in the Spatial KG. We use a VLM-based conflict resolver to address this problem. However, the more relationships provided to the conflict resolver, the more time it needs to update the KG. So we need to update relationships that are only related to the current situation. We design an algorithm that retrieves a sub-graph of KG that includes all vertices related to the current situation and both old and new relationships among them. We provide the sub-graph and new relationships to the conflict resolver. We need the conflict resolver to update the sub-graph based on information from the new relationships. The algorithm is shown in Algorithm 2.

To update G , we make use of the information provided by the step summarizer and query generator from the information preprocessor introduced in Section III-A. First, a pretrained VLM-based Relation Retriever extracts the latest spatial relationships $G_{\text{new}} = (V_{\text{new}}, E_{\text{new}})$ from the information provided by the step summarizer, which records high-level information in the current observation. Next, natural language queries (represented as $q \in Q$) (provided by the query generator) are used to retrieve relevant object vertices from G via cosine similarity search. We select the top n similar

vertices compared with the query. These vertices are represented by V_{similar} .

Spatial KG maintains the relationships among different objects, so if we want to retrieve spatial information from spatial KG, we need to search for other objects that are related to the objects we observed in the current observation. In this way, we not only remember objects we can see, but also know the spatial information of the objects we cannot see. So we choose the k-hop algorithm to expand V_{similar} using a K-hop neighborhood algorithm to capture contextually related objects. The K-hop algorithm is represented as $\text{K-hop}_k(V, G)$, which returns all vertices reachable within $\leq k$ hops from any vertices in V . The retrieved vertices are V_{expand} . We combine the vertices retrieved by cosine similarity and their K-hop neighbors to $V_{\text{retrieved}}$.

However, we need to resolve the conflict between new and old relationships. So $V_{\text{retrieved}}$ and the relationships among $V_{\text{retrieved}}$ is not enough. We need new vertices and relationships involved in the graph we provided to the VLM-based conflict resolver. To extract all vertices and relationships for the VLM-based conflict resolver and relationships, we not only need the relationships from KG (old information) and G_{new} , which represent new information. We need to connect old information and new information. To achieve this goal, we merge G_{new} to G , which add new edges and vertices to G . We denote the merged KG as G_{union} . In G_{union} , we mix out-of-date and latest information. Then, we extract an induced sub-graph of $V_{\text{merged}} = V_{\text{retrieved}} \cup V_{\text{new}}$ from G_{union} . As both vertices from old graph G and new graph is mixed in V_{merged} and both edges from old and new graph is in G_{union} , the retrieved induced sub-graph G_{local} contains all out-of-date and latest relationships among vertices that is related to current situation.

As G_{local} contains all out-of-date and latest relationships and those relationships may have conflicts, a VLM-based conflict resolver (represented as $\text{ResolveConflict}(\cdot)$) is designed to resolve conflicts in G_{local} , and make sure that the relationship is the latest. The conflict resolver will take in G_{new} and G_{local} , where G_{local} is the graph waiting for update and G_{new} provide update signal. The VLM-based conflict resolver will perform necessary updates such as adding vertices or inserting,

Algorithm 1 RoboMemory Execution Process

Require: Task description $\mathcal{T}$, Initial observation $\mathcal{O}_0$, Max steps T_{max}

Require: Modules: Step Summarizer $\mathcal{S}$, Query Generator $\mathcal{Q}$; Memory $\mathcal{U}$, $\mathcal{R}$; Planner $\mathcal{P}$; Critic $\mathcal{C}$; Executor $\mathcal{E}$

```
1: Initialize: Global step  $t \leftarrow 0$ , Memory  $M_t \leftarrow \emptyset$ 
2: Initial Perception:
3:  $s_t, q_t \leftarrow \mathcal{SQ}(\mathcal{O}_t)$  {call the step summarizer and query generator in parallel}
4:  $M_t \leftarrow \mathcal{U}(M_t, s_t)$  {Initialize Memory with first observation}
5: while  $t < T_{max}$  and Task  $\mathcal{T}$  not completed do
6:   Retrieval Phase:
7:    $r_t \leftarrow \mathcal{R}(M_t, q_t)$  {Parallel retrieval from  $L$  memory modules}
8:   Planning Phase:
9:    $\mathbf{A} \leftarrow \mathcal{P}(r_t, \mathcal{O}_t, \mathcal{T})$  {Generate action sequence  $\mathbf{A} = [a_1, a_2, \dots, a_K]$ }
10:  Execution Phase (Closed-Loop):
11:  for  $k = 1$  to  $|\mathbf{A}|$  do
12:    Let  $a_k$  be the current action to execute
13:     $execute\_flag \leftarrow \text{False}$ 
14:    if  $k = 1$  then
15:       $execute\_flag \leftarrow \text{True}$  {Skip Critic for the first step to avoid infinite loops}
16:    else
17:      {Re-evaluate context for subsequent steps}
18:       $r_{curr} \leftarrow \mathcal{R}(M_t, q_{curr})$ 
19:      if  $\mathcal{C}(a_k, r_{curr}, \mathcal{O}_t, \mathcal{T})$  is True then
20:         $execute\_flag \leftarrow \text{True}$ 
21:      else
22:        break {Critic rejects action; trigger re-planning}
23:      end if
24:    end if
25:    if  $execute\_flag$  is True then
26:       $\mathcal{O}_{t+1} \leftarrow \mathcal{E}(a_k)$  {Execute action via low-level executors}
27:       $t \leftarrow t + 1$ 
28:      Memory Update (Perception  $\rightarrow$  Memory):  $s_t, q_t \leftarrow \mathcal{SQ}(\mathcal{O}_t)$  {Generate query and summary in parallel}
29:       $M_t \leftarrow \mathcal{U}(M_{t-1}, s_t)$  {Parallel update of all modules}
30:    end if
31:  end for
32: end while
```

deleting, or modifying relationships in G_{local} based on G_{new} . The reconciled subgraph is then merged back into G , and any vertices that have lost all connections to other vertices during the update are pruned.

This design offers two key advantages: (1) Efficiency via localized updates: By restricting modifications to a context-relevant subgraph, we significantly reduce the number of relationships processed per update. Since VLMs struggle with reasoning over large sets of relationships, this constraint substantially improves both the efficiency and effectiveness of VLM-based KG updates. (2) Dynamic adaptability: The system continuously maintains up-to-date spatial knowledge, enabling agents to operate robustly in dynamic real-world environments.

2) Example of Dynamic Spatial Memory Update process:

In RoboMemory’s Spatial Memory, the KG is dynamically constructed during environment exploration. As illustrated in Figure 8, we demonstrate the progressive expansion of the

KG in Spatial Memory as the agent navigates through the environment. The figure indicates a continuous growth in the number of both vertices and edges of the KG as exploration progresses.

Notably, the KG undergoes dynamic updates through RoboMemory’s environmental interactions. For example, the initial KG state displays the relation “I am near the apple”. But as the agent picks up the apple in the third step, in the fourth KG, the relationship becomes “I hold the apple”. This demonstrates RoboMemory’s capability for dynamic KG maintenance and expansion.

By querying this KG, the Planner-Critic module gains access to rich spatial information, empowering RoboMemory with robust spatial memory capabilities that significantly enhance its performance in EmbodiedBench environments.

D. Proof of Dynamic Spatial Memory Update Algorithm

Theorem 1 (Upper Bound on K-hop Vertex Extraction in Directed Graphs). *Let $G = (V, E)$ be a finite directed graph*

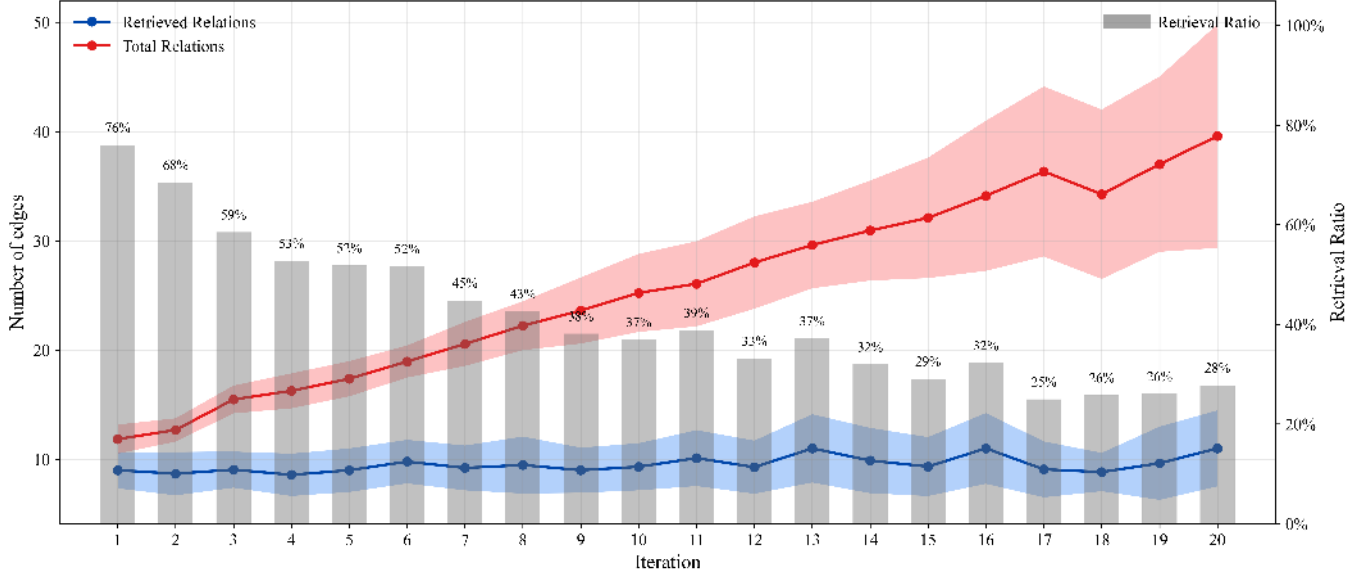


Fig. 7: Average relationships related to update in spatial memory in each step.

Algorithm 2 Retrieval-based Incremental Knowledge Graph Update Algorithm

Require: New spatial knowledge graph $G_{\text{new}} = (V_{\text{new}}, E_{\text{new}})$, main spatial knowledge graph $G = (V, E)$, queries $q \in Q$, entity & query embeddings $\mathcal{E} : V \cup Q \rightarrow \mathbb{R}^d$, maximum number of retrieved vertices n , maximum k hops k , vlm-base conflict resolver $\text{ResolveConflict}(\cdot)$

Ensure: Updated consistent knowledge graph G'

- 1: $V_{\text{similar}} \leftarrow \bigcup_{q \in Q} \text{TopK}_n(\{v \in V \mid \text{cosine_sim}(\mathcal{E}(q), \mathcal{E}(v))\})$ {For each query entity q , retrieve its top- n most similar vertices in G by cosine similarity of embeddings; take the union over all $q \in Q$.}
 - 2: $V_{\text{expand}} \leftarrow \text{K-hop}_k(V_{\text{similar}}, G)$ {all nodes within k hops from any node in V_{similar} }
 - 3: $V_{\text{retrieved}} \leftarrow V_{\text{similar}} \cup V_{\text{expand}}$
 - 4: $V_{\text{merged}} \leftarrow V_{\text{retrieved}} \cup V_{\text{new}}$
 - 5: $G_{\text{union}} \leftarrow (V \cup V_{\text{new}}, E \cup E_{\text{new}})$ {Combine the main graph and new observations into a unified graph.}
 - 6: $G_{\text{local}} \leftarrow \text{InducedSubgraph}(V_{\text{merged}}, G_{\text{union}})$ {Extract the subgraph induced by V_{merged} , containing all old and new edges among these nodes.}
 - 7: $G_{\text{updated}} \leftarrow \text{ResolveConflict}(G_{\text{local}}, G_{\text{new}})$ {based on G_{new} , VLM update the relationship among different vertices in G_{local} }
 - 8: $G' \leftarrow (G \setminus G_{\text{local}}) \cup G_{\text{updated}}$ {Replace the old subgraph in G with the conflict-resolved updated subgraph.}
 - 9: Remove isolated vertices from G'
 - 10: **return** G'
-

with maximum out-degree $D \geq 1$, and let $\mathcal{S} \subseteq V$ be a set of M source vertices. Define the K-hop neighborhood $\mathcal{N}_K(s)$ of a vertex $s \in \mathcal{S}$ as the set of vertices reachable from s via directed paths of length at most K . Then the total number of distinct vertices in the union of all K-hop neighborhoods,

$$\mathcal{N}_K(\mathcal{S}) = \bigcup_{s \in \mathcal{S}} \mathcal{N}_K(s),$$

Satisfies the following upper bound:

$$|\mathcal{N}_K(\mathcal{S})| \leq \begin{cases} M \cdot \frac{D^{K+1} - 1}{D - 1}, & \text{if } D > 1, \\ M \cdot (K + 1), & \text{if } D = 1. \end{cases}$$

Proof: For any vertex $s \in \mathcal{S}$, the number of distinct vertices reachable from s within i hops is at most D^i , assuming

the worst-case scenario where each vertex encountered has the maximum out-degree D , and all neighbors are distinct and non-overlapping.

Thus, the size of the K-hop neighborhood of a single vertex satisfies:

$$|\mathcal{N}_K(s)| \leq \sum_{i=0}^K D^i = \begin{cases} \frac{D^{K+1} - 1}{D - 1}, & \text{if } D > 1, \\ K + 1, & \text{if } D = 1. \end{cases}$$

Since there are M such source vertices and assuming no overlaps between their K-hop neighborhoods (worst case), the union size satisfies:

$$|\mathcal{N}_K(\mathcal{S})| \leq M \cdot |\mathcal{N}_K(s)|.$$

Substituting the bound on $|\mathcal{N}_K(s)|$ gives the result. ■

Since EB-ALFRED and EB-Habitat provide comprehensive high-level action APIs, we do not employ the VLA-Based Low-Level Executor in these environments. Instead, we utilize the built-in low-level controllers from EmbodiedBench.

2) *Additional Settings for Baselines: EB-ALFRED and EB-Habitat.* For our single VLM-Agent baseline, we utilized the reported results from the EmbodiedBench paper to establish a consistent benchmark, where the agent relies on a basic interaction history as its memory module. For other VLM frameworks, we replicated the experimental setups as described in both EmbodiedBench and the respective original papers. Crucially, to familiarize all baseline agents with the EmbodiedBench environment, we supplemented them with a few-shot example and a comprehensive catalog of actionable objects—applying the exact same conditions as those used for the single VLM-Agent benchmark.

For VLM-Agent Frameworks, we use the same VLMs (listed in Table I) to power all modules that require VLM control.

3) *Real-world experiments:* We construct a common kitchen scenario to evaluate the RoboMemory framework’s interactive environmental learning capabilities in real-world settings. Using Mobile ALOHA [17] as our physical robotic platform, we design three categories of tasks: (1) Pick up & put down: The agent must locate a specified object among all possible positions and place it at a designated location. This task tests the model’s basic object-searching and planning abilities. (2) Pick up, operate & put down: Building upon the first task, the agent must additionally perform operations such as heating or cleaning the object. This task requires longer-term planning, which is crucial in embodied environments. (3) Pick up, gather & put down: The agent must place specified objects into a movable container and then move the container to a target location. This task evaluates the agent’s understanding of object relationships, requiring it to remember the positions of at least two objects (the container and the target item) and their spatial relationship. For each type of task, we design 5 tasks. So our experiments include 15 long-term real-world tasks.

To adapt to the real-world setup, we define high-level action APIs similar to those in EB-ALFRED and EB-Habitat. Additionally, we train a VLA-based model to execute tasks according to our action APIs. The detailed action APIs are presented in Table IV.

For the low-level executor, we use one main camera and two arm-mounted cameras as input, each with a resolution of 640×480 . The temporal memory buffer length is set to 3.

In our experiments, we set the maximum steps per task to 25. We also provide an API for actively terminating tasks. Since real-world environments lack direct success/failure feedback, RoboMemory must autonomously determine task completion. To prevent excessively long task execution, we enforce termination after 25 steps if no success is achieved. A single main camera (640×480 resolution) records video during action execution as input for RoboMemory’s higher-level processing.

4) *Training Details of Low-Level Executor:* We use the π_0 model as our foundation model. We collected 1,040 data samples over 10 types of tasks for fine-tuning. We use LoRA fine-tuning to save resources during fine-tuning. The specific fine-tuning parameters and action types are given in Table V. For tasks involving both pick-up and place actions, we split these tasks into separate pick-up and place actions. These are then treated as two distinct data samples during training. The separation of pick-up and place action allows the VLA to carry an object in its hand. For training, we used a server with six A100-80GB GPUs. The total training time was 12 hours.

Besides, we use the built-in LiDAR SLAM system of the Mobile ALOHA robot base as the navigation action actuator. We define five typical navigation points, similar to EB-Habitat. We used SLAM to navigate between these navigation points.

5) *Hyperparameters of RoboMemory:* In this section, we describe the hyperparameter settings for the upper brain of RoboMemory: the information preprocessor and the Comprehensive Embodied Memory. Importantly, we use a unified set of hyperparameters across all experimental settings, including EB-ALFRED, EB-Habitat, and real-world deployments.

The information preprocessor consists of two components: a step summarizer and a query generator. Given multimodal inputs at each step, the step summarizer produces a single natural language description, while the query generator concurrently formulates 4 – 5 distinct natural language queries.

The Comprehensive Embodied Memory integrates four memory modules: Temporal, Spatial, Semantic, and Episodic Memory. The Temporal Memory is implemented as a fixed-size buffer with a maximum capacity of 4 entries. For Spatial Memory, during similarity-based retrieval, we first identify the top $N = 3$ most relevant vertices and then perform a K-hop graph traversal with $K = 2$. The Episodic Memory retrieves the top $N = 5$ most relevant past experiences for each query. The Semantic Memory maintains hierarchical summaries at both the action and task levels; during retrieval, it returns $N_s = 2$ action-level and $N_t = 2$ task-level summaries. Furthermore, memory updates (e.g., insertion, modification, or deletion) are applied only to the top $N_{\text{update}} = 10$ most relevant entries in the Semantic Memory to ensure efficiency and coherence.

F. Supplementary Examples for Qualitative Analysis

1) *Real World:* In Figure 9, we demonstrate an example of RoboMemory learning through trial and error in a real-world environment. Our task is “place a banana into the oven.” This task required RoboMemory to complete the objectives of finding the banana, picking it up, and transporting it to the oven. We observed that RoboMemory became stuck in an infinite loop during the first attempt. The banana was randomly placed on the “kitchen counter,” but RoboMemory overlooked this navigation target and remained trapped, exploring other navigation targets instead.

However, based on this bad attempt, the semantic memory summarized that the robot should not repeatedly search in locations where the “banana” could not be found. Meanwhile,

TABLE IV: Robot Action Command For different environments

Action Type	EB-ALFRED	EB-Habitat	Real World
Navigation	find(obj)	navigate(point)	navigate_to(point)
Pick Up Object	pick_up(obj)	pick(obj)	pick_up(obj)
Drop to Ground	drop()	—	—
Place to Receptacle	put_down()	place(rec)	put_down_to(rec)
Open Object	open(obj)	open(obj)	open(obj)
Close Object	close(obj)	close(obj)	close(obj)
Turn On	turn_on(obj)	—	turn_on(obj)
Turn Off	turn_off(obj)	—	turn_off(obj)
Slice Object	slice(obj)	—	—
Task Complete	—	—	task_complete()

TABLE V: Dataset statistics and training hyperparameters for robotic manipulation tasks.

Dataset Statistics		Training Configuration	
Action Type	#Episodes	Parameter	Value
Turning on/off faucet	142	Optimizer	AdamW
Picking up & Placing basket on counter	63	Batch size	32×6
Picking up & Placing basket in sink	72	Training steps	10,000
Picking up & Placing banana into basket	114	Learning rate	6.12×10^{-5}
Throwing bottle into trash bin	132	warm up step	500
Placing gum box on dish	120	LoRA Configuration	
Picking up & Placing cup on plate	51	rank	16
Picking up & Placing dish into sink	69	α	16
Throwing paper ball into trash bin	135	Resource Usage	
Open/close oven	142	GPU	A100-80GB $\times$ 6
Total episodes	1040	Training time	12 hours

the episodic memory recorded what RoboMemory had done and the outcomes during the first attempt. Based on the information provided by semantic and episodic memory, in the second attempt, RoboMemory recognized that it had not previously tried navigating to the “kitchen counter.” After attempting this, it successfully completed the task. This example illustrates the role of RoboMemory’s long-term memory.

We also provide an example that completes the task in the first attempt. The example is shown in Figure 10. This example demonstrates that the RoboMemory has the ability to handle some relatively complex tasks in the real world. The task in this example is “Place a box of gum into the basket and put the basket on the kitchen counter”. Because two objects in different positions are involved in this task, RoboMemory has to memorize the position of at least one object to achieve the goal. With the help of the spatial memory, RoboMemory completes the task successfully.

2) *EB-ALFRED*: We select three examples in EB-ALFRED to show the errors that RoboMemory may encounter and the reasons why or why not RoboMemory can achieve the goal.

Successful example. We select a successful example to show how RoboMemory performed in the EB-ALFRED environment. The example trajectory is shown in Figure 11.

The task of this example is “set a plate with a spoon on it on the kitchen table”. However, in step 10, the Planner seems to ignore the temporal information from memory modules. RoboMemory thinks that it still needs to pick up the spoon (even though it has already placed a spoon in the plate). However, with the help of the critic, it finally becomes aware that picking up another spoon is redundant, so RoboMemory

goes back to the current trajectory and successfully completes the task at the end.

In this example, RoboMemory successfully overcame the hallucination and eventually achieved the goal. This example demonstrates that the critic module can help RoboMemory to overcome error cases.

Failed example. We demonstrate a representative example of the Critical Error. The example trajectory is shown in Figure 12. In this example, the task involves slicing and heating a tomato and moving the heated tomato slice to the trash can. Initially, RoboMemory successfully sliced the tomato with a knife. But when the planner plans the whole sequence, it forgets to drop the knife before picking up the tomato (this is necessary because in EB-ALFRED, the robot can only hold one object at a time). The critic and the planner should notice this situation and ask the critic to replan, as RoboMemory failed to pick up a tomato slice. However, the critic module ignores this issue, and thus, after it heats the knife instead of a tomato slice, it stacks in an infinite loop.

Besides, we provide another example demonstrating a representative failure caused by inaccurate action planning. The example trajectory is shown in Figure 13. In the trajectory, RoboMemory is asked to place two CDs into the drawer. However, at step 6, the robot failed to select correct CD object. In this experiment, RoboMemory has already put CD_2 into the drawer, but it keeps picking up CD_2 even though the memory has clearly indicated that CD_2 has already been put down. So we classify this as inaccurate action error. This indicates that the planner failed to comprehensively integrate information from both the memory and information-gathering

modules, resulting in inaccurate action planning.

Fig. 9: Case that a task is failed, but the experience can help RoboMemory to succeed in the next try.

Fig. 10: Case that a task is successful.

Fig. 11: Case that a task is successful with the help of the critic and spatial memory modules.

Fig. 12: Case that a task fails in an infinite loop because the critic module failed to stop the agent when its planned action is no longer suitable.

Fig. 13: Case that a task fails in an infinite loop because of inaccurate action planning.